LQR Control of Furuta Pendulum

Touchton, Noah
Section 12105 Date 12/3/25

***Abstract*— This experiment investigates closed-loop control of a Furuta pendulum using manually tuned gains and model-based Linear Quadratic Regulator (LQR) control. The platen and pendulum angles were measured, filtered, and differentiated to compute state feedback for a five-state controller. Manual tuning stabilized the platen but could not suppress pendulum motion until pendulum feedback was added. LQR gains derived from system dynamics provided comparable or improved tracking with lower effort. Adjusting Q and R demonstrated tradeoffs between pendulum suppression, tracking, and motor aggressiveness. Performance was evaluated using RMS angle and command metrics.**

***Index Terms*—** Bryson's Rule, Furuta pendulum, LQR control, Pendulum stabilization

I. Introduction

THE objective of this lab is to control a platen pendulum system using an LQR controller. By measuring the position and velocity of both the platen and the pendulum, the errors can be calculated and fed into a feedback loop to determine the input. Only the platen has a motor, so the dynamics of the system need to be evaluated to control both rotations.

Furuta Pendulum

The Furuta pendulum is a dynamic system that consists of both a platen and a pendulum where a motor controls the platen. In this lab, both the platen angle and pendulum angle are measured, and the angle signals are filtered, and the derivatives are taken to find the angular velocities. When the pendulum is inverted this is an unstable system which requires control. The hardware is set up to use a feedback loop to determine the input based on the current value of the states. Knowing the resistance of the motor and calculating the voltages based on the input command, the current could be calculated.

LQR Control

Linear-quadratic regulator control aims to weigh the costs of the inputs and outputs of a system to generate the ideal input. The system is modeled as a state space model as shown in (1). Where x is a matrix of the states and u is the input into the system. The control theory is derived from the optimization problem shown in (2).

$$\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\boldsymbol{u} \tag{1}$$

$$J = \int_0^\infty (\boldsymbol{x}^T Q\boldsymbol{x} + \boldsymbol{u}^T R\boldsymbol{u})\, dt \tag{2}$$

J represents the cost, and LQR attempts to find gain values that minimize J based on a chosen Q and R matrix. The Q matrix must be positive semidefinite, and the R matrix must be positive definite which ensures that both cost terms are always positive. Since it is an integral accumulates positive values over time, the controller chooses input values that drive the state to zero to minimize the integral. In general, a large Q and small R penalize state error more than input effort, producing faster and more aggressive control. A larger R has the opposite effect, discouraging large inputs and yielding a slower but smoother response. This can be useful in situations where the input is limited such as fuel on a spacecraft. A small input that uses little fuel is much more "valuable" than reaching the target destination as quickly as possible. Q and R are both matrices so "value" here is used informally. More precisely, the magnitude and eigenvalues of the matrix determine the optimization trade-offs.

Q is usually chosen to be a diagonal matrix and is a square matrix the same size as the state matrix. This means there is a constant that pairs with each state which allows each state can be given a value based on how important it is for that value to be minimized. In the context of the pendulum, it may be determined that the pendulum angle error being zero is the most critical so the value that lines up with that state should be the largest. Looking at the optimization equation in (1), this allows control over each state by choosing a proper Q matrix.

R is a column matrix corresponding to the number of inputs of the system. This mean each input can be given a weight of how "expensive" it is.

$$A^T P + PA - PBR^{-1}B^T P + Q = 0 \tag{3}$$

$$K = R^{-1}B^T P \tag{4}$$

To find the K matrix from this optimization problem, the Continuous Algebraic Riccati Equation shown in (3) can be solved for P which is used to find K as shown in (4). This takes into account the dynamics of the system using the A and B matrix to come up with control specific to the system. This will generate a matrix with the same number of columns as states and the same number of rows as inputs.

$$\boldsymbol{u} = -K\boldsymbol{x} \tag{5}$$

The error in the state is multiplied by the calculated -K matrix as shown in (5) to find the ideal input.

States

There were 5 states used in the calculations for this lab. The states were platen angle, pendulum angle, platen velocity, pendulum velocity, and current. Current was included as the fifth state because the DAQ commands voltage indirectly, making current the true input. The full nonlinear dynamics of the Furuta pendulum have been derived using both Lagrangian and Newton–Euler methods, then they were linearized, providing a validated foundation for our state-space model and control design. [2]

II. Methods

Hardware

This lab uses a platen with a slip ring controlled by a motor with a motor driver. The driver has a built-in angle sensor to measure the position. The platen is connected to a swinging pendulum that uses an encoder to track the angle of the pendulum. A DAQ was connected to the motor driver and was able to send and receive via LabVIEW.

Software

A custom LabVIEW VI was developed to implement controllers. The VI contains a signal generator as described in [1]. The signal generator serves as the desired angular position, which when the measured angular position is subtracted from, creates an error signal. This signal is fed into the controller to create a feedback loop. The errors are multiplied by the calculated gain constants to determine the input signal to the motor.

In MATLAB, the A and B matrices calculated in Lab 3 were used to solve (3) and (4) for the K matrix given a Q and R matrix. MATLAB's built in lqr function is able to streamline this process and easily calculate K. Bryson's rule was used to find the starting points for the Q and R weights. Bryson's rule says that the diagonal entries of Q and R should be equal to the inverse of the square of the maximum accepted value of the state or input as shown in (6,7).

$$\boldsymbol{Q_{ii}} = \frac{\mathbf{1}}{(\boldsymbol{max\ acceptable\ value\ of\ state\ i})^{\mathbf{2}}} \tag{6}$$

$$\boldsymbol{R_{jj}} = \frac{\mathbf{1}}{(\boldsymbol{max\ acceptable\ value\ of\ input\ j})^{\mathbf{2}}} \tag{7}$$

A python script was used to analyze all of the data. The python script parses through all of the data and finds the RMS for the two angles and the command. These can be used the quantify how good the controller was.

Procedure

First, the pendulum was installed but taped so that it would not rotate. The square wave amplitude and frequency were set to 60 degrees and 0.25hz. K_1 and K_3 correspond to the platen angle and platen velocity, and these values were tuned to have the platen angle smoothly follow the square wave input. Once the platen reached the target amplitude with little overshoot and an acceptable rise time the K values were saved. This data was recorded and then the tape was taken off the pendulum, and the data was recorded again when the pendulum could swing freely.

Next, K_2 and K_4, which correspond to the pendulum angle and velocity were tuned. One important thing to note is the K_2 and K_4 must be inverted to capture the correct dynamics of the non-inverted pendulum. This is because the A matrix assumes the pendulum is in the up position and the down position has an opposite error sign. These values were increased from zero until the pendulum angle stayed as close to zero as possible throughout the sweep.

Next, the LQR method discussed earlier was used to calculate theoretical K values and those were implemented into the system. Bryson's rule was used as a starting point, and the Q and R weights were modified based on the system response. For example, the initial guess of Bryson's rule made R very small which led to oversaturating the motor, this was modified to scale the numbers down.

Next, the Q matrix was modified to attempt to force the pendulum to be even smaller. Q_{22} was increased by decreasing the max acceptable value of state 2. The new gains were tested and data was collected.

Finally, R was manipulated by a factor of 0.1 and 10 to see how the system would respond. New K values were computed and implemented. The data was collected for these trials.

Root Mean Square

Root Mean Square (RMS) is a statistical measure that quantifies the magnitude of a varying signal. The equation for how it is calculated is shown in (8) as described in [3]. In the context of control signals, the RMS represents the average deviation between the desired and actual response. A low RMS means there is low error in the control method.

$$X_{RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(X_i)^2} \tag{8}$$

III. Results

Pendulum Down Response

The rms for the platen angle, the pendulum angle, and the command were calculated for the manually tuned K_1 and K_3 and taped pendulum. The same values were computed when the pendulum was free to rotate as well. These values are presented in Table I. The K values found from manual tuning are found in Table II.

TABLE I
PLATEN MANUALLY TUNED RMS VALUES

Taped	RMS θ_1 (deg)	RMS θ_2 (deg)	RMS Command
Yes	29.706	0.690	3.173
No	29.743	591.922	3.157

TABLE II
PLATEN MANUALLY TUNED K VALUES

K_1	K_2	K_3	K_4	K_5
4.0	0.0	0.5	0.0	0.0

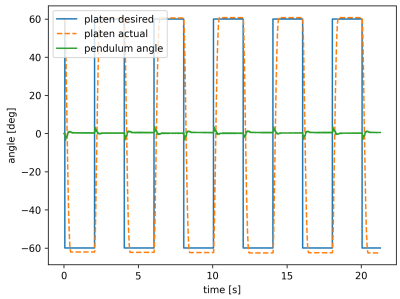

Fig 1. This shows the two measured angles plus the desired platen angle for the taped system.

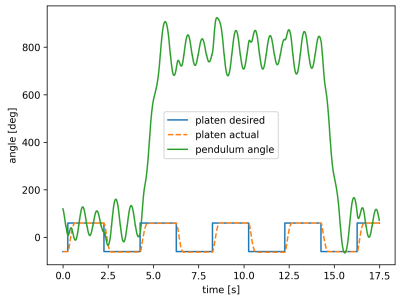

Fig. 2 This shows the two measured angles plus the desired platen angle for the untaped system.

Next, the K_2 and K_4 gain constants were tuned and implemented. The rms for the platen angle, the pendulum angle, and the command were calculated for the system and these values are presented in Table III. The K values found from manual tuning are found in Table IV.

TABLE III
PENDULUM MANUALLY TUNED RMS VALUES

RMS θ_1 (deg)	RMS θ_2 (deg)	RMS Command
53.351	9.611	2.596

TABLE IV
PENDULUM MANUALLY TUNED K VALUES

K_1	K_2	K_3	K_4	K_5
4.0	-15.0	0.5	-10	0.0

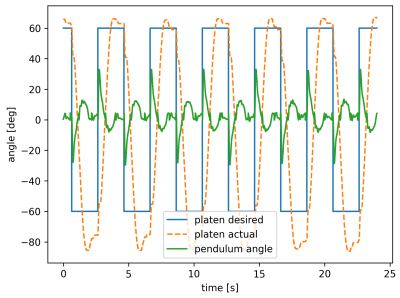

Fig 3. This shows the two measured angles plus the desired platen angle for the manually tuned system.

Then, LQR was used to determine the K values. The rms for the platen angle, the pendulum angle, and the command were calculated for the system and these values are presented in Table V. The K values found from LQR tuning are found in Table VI. The Q and R matrices used are shown in (9,10). These values were found using Bryson's rule.

TABLE V
LQR TUNED RMS VALUES

RMS θ_1 (deg)	RMS θ_2 (deg)	RMS Command
49.577	10.725	2.471

TABLE VI
LQR TUNED K VALUES

K_1	K_2	K_3	K_4	K_5
3.8217	-10.3302	0.4368	-0.0336	0.3446

$$\boldsymbol{Q} = \begin{bmatrix} \mathbf{0.912} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{2.682} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0.101} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0.113} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0.015} \end{bmatrix} \quad (9)$$

$$\boldsymbol{R} = [\mathbf{0.0625}] \quad (10)$$

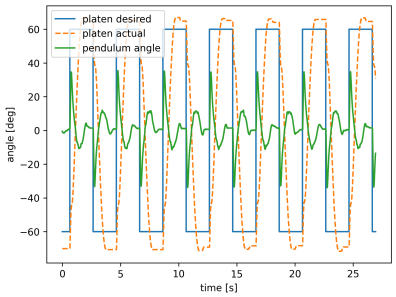

Fig 4. This shows the two measured angles plus the desired platen angle for the LQR tuned system.

Then, the LQR was adjusted to add more weight to the pendulum angle. Increasing the value of Q_{22} will encourage the system to prioritize getting that state to zero. Q_{22} was increased to 10.730 and the rest of the weights remained the same. The rms for the platen angle, the pendulum angle, and the command were calculated for the system and these values are presented in Table VII. The K values found from LQR tuning are found in Table VIII.

TABLE VII
Q_{22} RETUNED RMS VALUES

RMS θ_1 (deg)	RMS θ_2 (deg)	RMS Command
49.651	12.151	3.000

TABLE VIII
Q22 RETUNED K VALUES

K_1	K_2	K_3	K_4	K_5
3.8217	-14.091	0.5259	-0.041	0.369

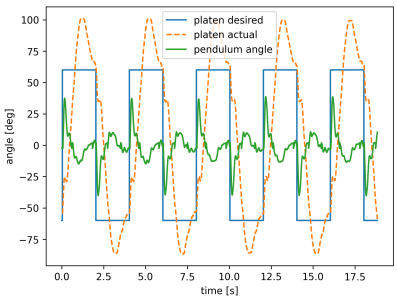

Fig 5. This shows the two measured angles plus the desired platen angle for the adjusted Q tuned system.

Lastly, the R value was both multiplied and divided by 10. This directly changes how much effort the motor puts into the system. When R is decreased it allows more input, and this caused the motor to move so fast that the pendulum flew off the platen very quickly. This was repeated many times and failed every time. The rms for the platen angle, the pendulum angle, and the command were calculated for the system and these values are presented in Table IX. The K values found from LQR tuning are found in Table X.

TABLE IX
R RETUNED RMS VALUES

R multiplier	RMS θ_1 (deg)	RMS θ_2 (deg)	RMS Command
0.1	105.302	31.572	37.271
10	52.773	2.5669	1.079

TABLE VIII
R RETUNED K VALUES

R multiplier	K_1	K_2	K_3	K_4	K_5
0.1	12.085	-43.802	2.588	-1.909	2.447
10	1.208	-3.862	0.089	0.102	0.069

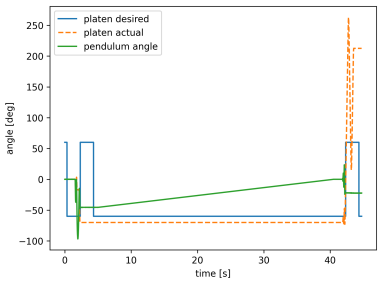

Fig 6. This shows the two measured angles plus the desired platen angle for the minimized R tuned system.

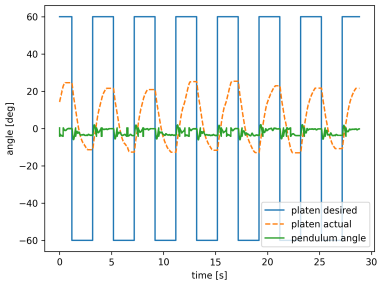

Fig 7. This shows the two measured angles plus the desired platen angle for the amplified R tuned system.

IV. DISCUSSION

The primary goals of this lab were to characterize the Furuta pendulum system and evaluate manually tuned and LQR tuned feedback loop controllers on their performance. The platen and pendulum angles were measured, filtered, and differentiated to obtain velocities. Through various manual tuning methods and LQR weight adjustments different sets of K gains were generated and tested on the system. Each configuration was evaluated using the RMS of both angles and the command for the duration of the test.

The first task was manually tuning the K values that related to the platen. The platen was taped to remove the swinging forces. The RMS for the platen angle was relatively small which makes sense for a step input since the platen will instantly have a large error to overcome. The pendulum RMS makes perfect sense being near 0 since it was not moving. Once the tape was removed the platen tracking generally stayed the same, but the pendulum was swinging uncontrollably leading to a very large RMS. This proves that only taking the platen error into account is not enough to control the pendulum.

The second task required the first four gains to be tuned manually. This ignores the current gain but saw vast improvement in the RMS of the pendulum angle. The pendulum did a much better job at staying near zero at the cost of decreased performance in the tracking of the platen. The system could not aggressively move the platen to the desired location without exciting the pendulum.

The third task used LQR to more precisely choose the gains rather than just playing around with them until the system responded well. It allowed choice as to which of the states is most important to control and used the dynamics of the system to relate the gains to each other. This method produced very similar results to the manually tuned method demonstrating that it is an effective way to tune a controller. The pendulum was slightly less accurate, but the platen was slightly more accurate. These values could be further tuned using the weights if desired. The Q and R values were initialized by Bryson's rule, which generally did a good job at getting good starting values and only needed small tweaks.

When Q_{22} was modified to put more weight on θ_2 being smaller and interesting side effect happened. The RMS for θ_2 and the RMS for the command both increased. This shows that the system tried to excite the motor more to drive θ_2 to zero, but this ended up just exciting the angle more. This gives a good example of hardware or dynamic modeling errors. There has to be a balance between input and state. For example, if I need to move the platen 2 degrees, the fastest way to do this is to send 12V to the motor but the hardware is not capable of stopping the motor quick enough so it will overshoot. This shows the balance of input and output when designing a controller.

Scaling R vastly changed the output of the system. When R got smaller this meant that input was less "valuable" and caused the motor to react extremely aggressively, so aggressively, such that the pendulum refused to stay attached which explains the weird data in Fig 6. Besides that, it was still very poor and would react so strongly the pendulum would just swing around and around. The platen would also constantly overshoot the target. The command RMS is significantly larger than the others. When R increased this led to improved performance on the pendulum angle since it was moving very slow, but the platen was not able to make it to its desired destination in time.

Finally, the hardware performance could be improved by having a more accurate encoder and if the piece that attaches the pendulum to the plate was stronger. I found that the pendulum angle had a lot of noise in it and led to very large magnitude for velocity. Also, the pendulum connector was extremely weak and would break often leading to lower motor input for it to not break.

REFERENCES

[1] N. Touchton, "Pendulum Hardware Constants," EML 4314C Lab 3 Report, University of Florida, Nov. 2025.
[2] B. S. Cazzolato, "On the dynamics of the Furuta pendulum," Math. Probl. Eng., vol. 2011, Article ID 528341, 10 pp., May 2011, doi:10.1155/2011/528341.
[3] "EML 4314C – Fall 2025 Lab 2 Assignment," University of Florida, 2025.